Dots!

While this method of drawing the dots works for now, there are a few issues we should address.

- We're adding a level of nesting, which makes our code harder to follow.
- If we run this function twice, we'll end up drawing two sets of dots. When we start updating our charts, we will want to draw and update our data with the same code to prevent repeating ourselves.

To address these issues and keep our code clean, let's handle the dots without using a loop.

Data joins

Scratch that last block of code. D3 has functions that will help us address the above problems.

We'll start off by grabbing all `<circle>` elements in a **d3 selection object**. Instead of using d3.selection's `.select()` method, which returns one matching element, we'll use its `.selectAll()` method, which returns an array of matching elements.

```
const dots = bounds.selectAll("circle")
```

This will seem strange at first — we don't have any dots yet, why would we select something that doesn't exist? Don't worry! You'll soon become comfortable with this pattern.

We're creating a d3 selection that is aware of what elements already exist. If we had already drawn part of our dataset, this selection will be aware of what dots were already drawn, and which need to be added.

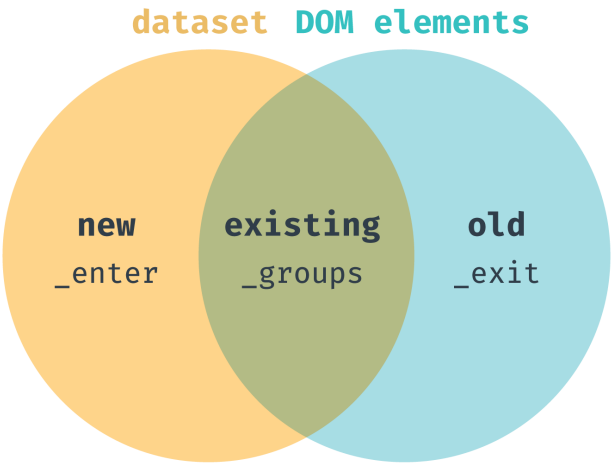
To tell the selection what our data look like, we'll pass our dataset to the selection's `.data()` method.

```
const dots = bounds.selectAll("circle")
    .data(dataset)
```

When we call `.data()` on our selection, **we're joining our selected elements with our array of data points**. The returned selection will have a list of **existing elements**, **new elements** that need to be added, and **old elements** that need to be removed.

We'll see these changes to our selection object in three ways:

- our selection object is updated to contain any overlap between existing DOM elements and data points
- an **_enter** key is added that lists any data points that don't already have an element rendered
- an **_exit** key is added that lists any data points that are already rendered but aren't in the provided dataset



join schematic

Let's get an idea of what that updated selection object looks like by logging it to the console.

```
let dots = bounds.selectAll("circle")
console.log(dots)
dots = dots.data(dataset)
console.log(dots)
```

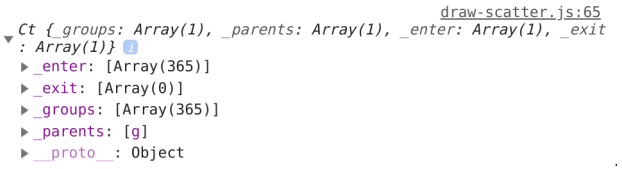
Remember, the currently selected DOM elements are located under the `_groups` key. Before we join our dataset to our selection, the selection just contains an empty array. That makes sense! There are no circles in **bounds** yet.

```
▼ Selection {_groups: Array(1), _parents: Array(1)}
  ▼ _groups: Array(1)
    ▶ 0: NodeList []
      length: 1
    ▶ __proto__: Array(0)
  ▶ _parents: [g]
  ▶ __proto__: Object
```

empty selection

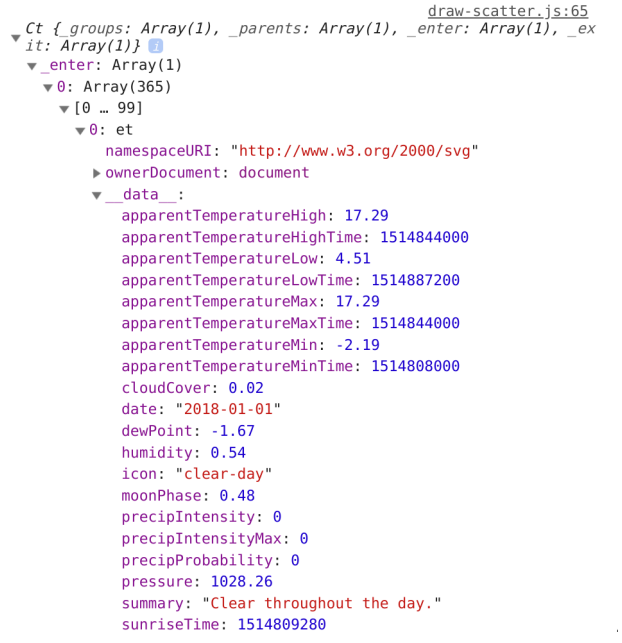
However, the next selection object looks different. We have two new keys: `_enter` and `_exit`, and our `_groups` array has an array with 365 elements — the number of

data points in our dataset.



.data selection

Let's take a closer look at the `_enter` key. If we expand the array and look at one of the values, we can see an object with a `__data__` property.



.data selection EnterNode

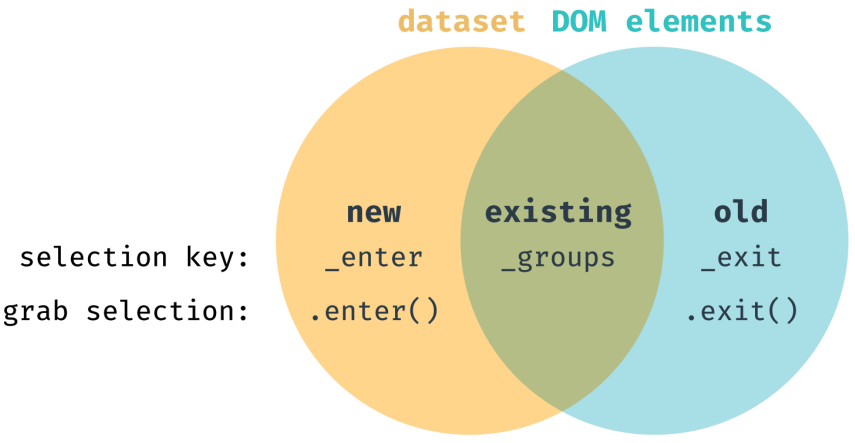
> For the curious, the `namespaceURI` key tells the browser that the element is a SVG element and needs to be created in the "http://www.w3.org/2000/svg" namespace (SVG), instead of the default "http://www.w3.org/1999/xhtml" namespace (XHTML).

If we expand the `__data__` value, we will see one our data points.

Great! We can see that each value in `_enter` corresponds to a value in our dataset. This is what we would expect, since all of the data points need to be added to the DOM.

The `_exit` value is an empty array — if we were removing existing elements, we would see those listed out here.

In order to act on the **new** elements, we can create a d3 selection object containing just those elements with the `enter` method. There is a matching method (`exit`) for **old** elements that we'll need when we go over transitions in **Chapter 4**.



join schematic with selection calls

Let's get a better look at that new selection object:

```
const dots = bounds.selectAll("circle")
    .data(dataset)
  .enter()
console.log(dots)
```

This looks just like any d3 selection object we've manipulated before. Let's append one `<circle>` for each data point. We can use the same `.append()` method we've been using for single-node selection objects and d3 will create one element for each data point.

```
const dots = bounds.selectAll("circle")
    .data(dataset)
  .enter().append("circle")
```

When we load our webpage, we will still have a blank page. However, we will be able to see lots of new empty `<circle>` elements in our **bounds** in the **Elements** panel.

Let's set the position and size of these circles.

```
const dots = bounds.selectAll("circle")
    .data(dataset)
  .enter().append("circle")
    .attr("cx", d => xScale(xAccessor(d)))
    .attr("cy", d => yScale(yAccessor(d)))
    .attr("r", 5)
```

We can write the same code we would write for a single-node selection object. Any attribute values that are functions will be passed each data point individually. This helps keep our code concise and consistent.

Let's make these dots a lighter color to help them stand out.

```
.attr("fill", "cornflowerblue")
```

Data join exercise

Here's a quick example to help visualize the data join concept. We're going to split the dataset in two and draw both parts separately. Temporarily comment out your finished dots code so we have a clear slate to work with. We'll put it back when we're done with this exercise.

Let's add a function called `drawDots()` that mimics our dot drawing code. This function will select all existing circles, join them with a provided dataset, and draw any new circles with a provided color.

```
function drawDots(dataset, color) {
  const dots = bounds.selectAll("circle").data(dataset)

  dots
    .enter().append("circle")
    .attr("cx", d => xScale(xAccessor(d)))
    .attr("cy", d => yScale(yAccessor(d)))
    .attr("r", 5)
    .attr("fill", color)
}
```

Let's call this function with part of our dataset. The color doesn't matter much — let's go with a dark grey.

```
drawDots(dataset.slice(0, 200), "darkgrey")
```

We should see some of our dots drawn on the page.

grey dots

After one second, let's call the function again with our whole dataset, this time with a blue color. We're adding a timeout to help distinguish between the two sets of dots.

```
setTimeout(() => {
  drawDots(dataset, "cornflowerblue")
}, 1000)
```

When you refresh your webpage, you should see a set of grey dots, then a set of blue dots one second later.

grey + blue dots

Each time we run `drawDots()`, we're setting the color of only **new** circles. This explains why the grey dots stay grey. If we wanted to set the color of all circles, we could re-select all circles and set their fill on the new selection:

```
function drawDots(dataset, color) {
  const dots = bounds.selectAll("circle").data(dataset)

  dots.enter().append("circle")
  bounds.selectAll("circle")
      .attr("cx", d => xScale(xAccessor(d)))
      .attr("cy", d => yScale(yAccessor(d)))
      .attr("r", 5)
      .attr("fill", color)
}
```

In order to keep the chain going, d3 selection objects have a `merge()` method that will combine the current selection with another selection. In this case, we could combine the new `enter` selection with the original `dots` selection, which will return the full list of dots. When we set attributes on the new merged selection, we'll be updating all of the dots.

```
function drawDots(dataset, color) {
  const dots = bounds.selectAll("circle").data(dataset)

  dots
    .enter().append("circle")
    .merge(dots)
      .attr("cx", d => xScale(xAccessor(d)))
      .attr("cy", d => yScale(yAccessor(d)))
      .attr("r", 5)
      .attr("fill", color)
}
```

`.join()`

Since d3-selection version 1.4.0[20], there is a new `.join()`[21] method that helps to cut down on this code. `.join()` is a shortcut for running `.enter()`, `.append()`, `.merge()`, and some other methods we haven't covered yet. This allows us to write the following code instead:

[20] https://github.com/d3/d3-selection/releases/tag/v1.4.0
[21] https://github.com/d3/d3-selection/#selection_join

```
function drawDots(dataset, color) {
  const dots = bounds.selectAll("circle").data(dataset)

  dots.join("circle")
    .attr("cx", d => xScale(xAccessor(d)))
    .attr("cy", d => yScale(yAccessor(d)))
    .attr("r", 5)
    .attr("fill", color)
}
```

While `.join()` is a great addition to d3, it's still beneficial to understand the `.enter()`, `.append()`, and `.merge()` methods. Most existing d3 code will use these methods, and it's important to understand the basics before getting fancy.

Don't worry if this pattern still feels new — we'll reinforce and build on what we've learned when we talk about transitions. For now, let's delete this example code, uncomment our finished dots code, and move on with our scatter plot!

Draw peripherals

Let's finish up our chart by drawing our axes, starting with the x axis.

We want our x axis to be:

- a line across the bottom
- with spaced "tick" marks that have...
- labels for values per tick
- a label for the axis overall

To do this, we'll create our axis generator using `d3.axisBottom()`, then pass it:

- our x scale so it knows what ticks to make (from the domain) and
- what size to be (from the range).

code/02-making-a-scatterplot/completed/draw-scatter.js

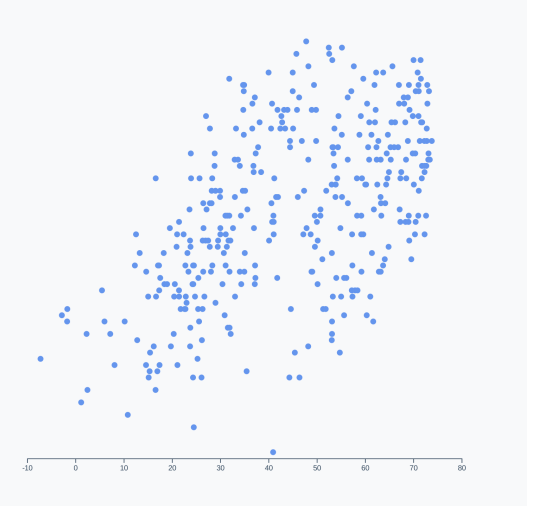
```
76  const xAxisGenerator = d3.axisBottom()
77    .scale(xScale)
```

Next, we'll use our `xAxisGenerator()` and call it on a new `g` element. Remember, we need to translate the x axis to move it to the bottom of the chart bounds.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
79  const xAxis = bounds.append("g")
80    .call(xAxisGenerator)
81      .style("transform", `translateY(${dimensions.boundedHeight}px)`)
```

When we render our webpage, we should see our scatter plot with an x axis. As a bonus, we can see how using `.nice()` on our scale ensures that our axis ends in round values.



Scatterplot with an x axis

Let's expand on our knowledge and create labels for our axes. Drawing text in an SVG is fairly straightforward - we need a `<text>` element, which can be positioned with an `x` and a `y` attribute. We'll want to position it horizontally centered and slightly above the bottom of the chart.

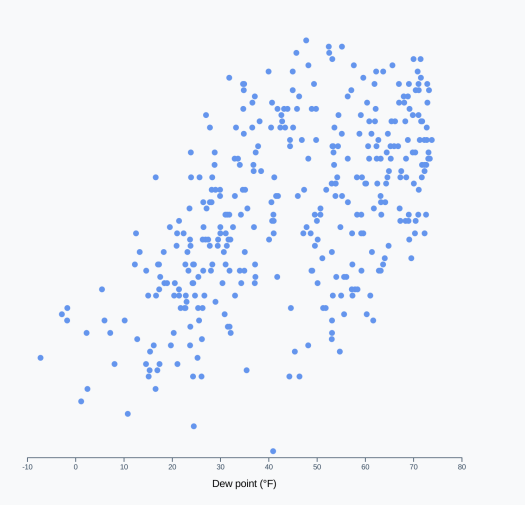
`<text>` elements will display their children as text — we can set that with our selection's `.html()` method.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
83  const xAxisLabel = xAxis.append("text")
84      .attr("x", dimensions.boundedWidth / 2)
85      .attr("y", dimensions.margin.bottom - 10)
86      .attr("fill", "black")
87      .style("font-size", "1.4em")
88      .html("Dew point (&deg;F)")
```

> We need to explicitly set the text fill to black because it inherits a fill value of none that d3 sets on the axis `<g>` element.

Great! Now we can see a label underneath our x axis.



Scatter plot with an x axis label

Almost there! Let's do the same thing with the y axis. First, we need an axis generator.

D3 axes can be customized in many ways. An easy way to cut down on visual clutter

is to tell our axis to aim for a certain number with the `ticks` method. Let's aim for 4 ticks, which should give the viewer enough information.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
90  const yAxisGenerator = d3.axisLeft()
91    .scale(yScale)
92    .ticks(4)
```

> Note that the resulting axis won't necessarily have exactly 4 ticks. D3 will take the number as a suggestion and aim for that many ticks, but also trying to use friendly intervals. Check out some of the internal logic in the **d3-array** code[a] — see how it's attempting to use intervals of 10, then 5, then 2?
>
> There are many ways to configure the ticks for a d3 axis — find them all in the docs[b]. For example, you can specify their exact values by passing an array of values to `.tickValues()`.
>
> [a] https://github.com/d3/d3-array/blob/master/src/ticks.js#L43
> [b] https://github.com/d3/d3-axis#axis_ticks

Let's use our generator to draw our y axis.

code/02-making-a-scatterplot/completed/draw-scatter.js

```
94  const yAxis = bounds.append("g")
95      .call(yAxisGenerator)
```

To finish up, let's draw the y axis label in the middle of the y axis, just inside the left side of the chart wrapper. d3 selection objects also have a `.text()` method that operates similarly to `.html()`. Let's try using that here.

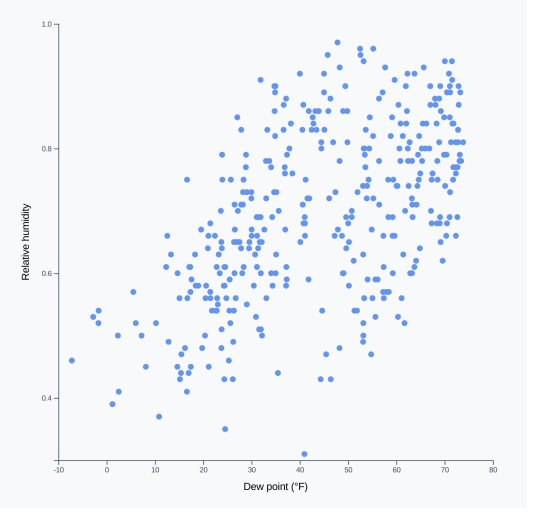
code/02-making-a-scatterplot/completed/draw-scatter.js

```
97    const yAxisLabel = yAxis.append("text")
98        .attr("x", -dimensions.boundedHeight / 2)
99        .attr("y", -dimensions.margin.left + 10)
100       .attr("fill", "black")
101       .style("font-size", "1.4em")
102       .text("Relative humidity")
```

We'll need to rotate this label to fit next to the y axis. To rotate it around its center, we can set its CSS property `text-anchor` to `middle`.

```
.style("transform", "rotate(-90deg)")
.style("text-anchor", "middle")
```

And just like that, we've drawn our scatter plot!



Finished scatterplot

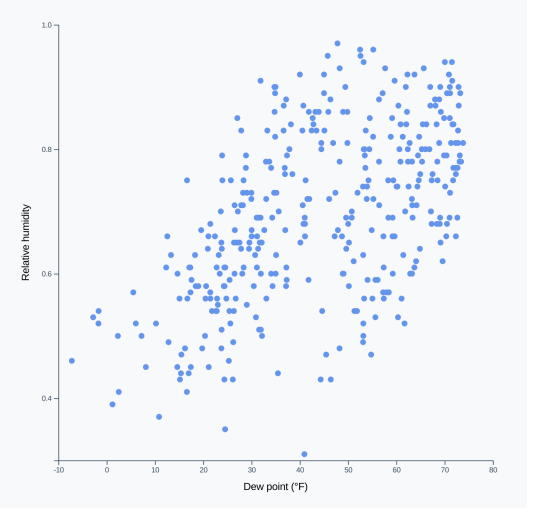
Initialize interactions

The next step in our chart-drawing checklist is setting up interactions and event listeners. We'll go over this in detail in **Chapter 5**.

Looking at our chart

Now that we've finished drawing our scatter plot, we can step back and see what we can learn from displaying the data in this manner. Without running statistical analyses (such as the Pearson Correlation Coefficient or Mutual Analyses), we won't be able to make any definitive statements about whether or not our metrics are correlated. However, we can still get a sense of how they relate to one another.

Looking at the plotted dots, they do seem to group around an invisible line from the bottom left to the top right of the chart.



Finished scatterplot

Generally, it seems like we were correct in guessing that a high humidity would likely coincide with a high dew point. We'll discuss the different types of patterns we might see in a scatter plot in Chapter 10.

Extra credit: adding a color scale

Remember when I said that we could introduce another metric? Let's bring in the amount of cloud cover for each day. In our dataset, each datapoint records the

cloud cover for that day. Let's show how the amount of cloud cover varies based on humidity and dew point by adding a color scale.

chart.js:6

(index)	Value
time	1514782800
summary	"Clear throughout the day."
icon	"clear-day"
sunriseTime	1514809280
sunsetTime	1514842810
moonPhase	0.48
precipIntensity	0
precipIntensityMax	0
precipProbability	0
temperatureHigh	18.39
temperatureHighTime	1514836800
temperatureLow	12.23
temperatureLowTime	1514894400
apparentTemperatureHigh	17.29
apparentTemperatureHighTime	1514844000
apparentTemperatureLow	4.51
apparentTemperatureLowTime	1514887200
dewPoint	-1.67
humidity	0.54
pressure	1028.26
windSpeed	4.16
windGust	13.98
windGustTime	1514829600
windBearing	309
cloudCover	0.02
uvIndex	2
uvIndexTime	1514822400
visibility	10
temperatureMin	6.17
temperatureMinTime	1514808000
temperatureMax	18.39
temperatureMaxTime	1514836800
apparentTemperatureMin	-2.19
apparentTemperatureMinTime	1514808000
apparentTemperatureMax	17.29
apparentTemperatureMaxTime	1514844000
date	"2018-01-01"

▶ Object

Our dataset

Looking at a value in our dataset, we can see that the amount of cloud cover exists at the key `cloudCover`. Let's add another data accessor function near the top of our file:

```
const colorAccessor = d => d.cloudCover
```

Next up, let's create another scale at the bottom of our **Create scales** step.

So far, we've only looked at linear scales that convert numbers to other numbers. Scales can also convert a number into a color — we just need to replace the **domain** with a range of colors.

Let's make low cloud cover days be light blue and very cloudy days dark blue - that's a good semantic mapping.

```
const colorScale = d3.scaleLinear()
    .domain(d3.extent(dataset, colorAccessor))
    .range(["skyblue", "darkslategrey"])
```

Let's test it out - if we log `colorScale(0.1)` to the console, we should see a color value, such as `rgb(126, 193, 219)`. Perfect!

> Choosing colors is a complicated topic! We'll learn about color spaces, good color scales, and picking colors in **Chapter 7**.

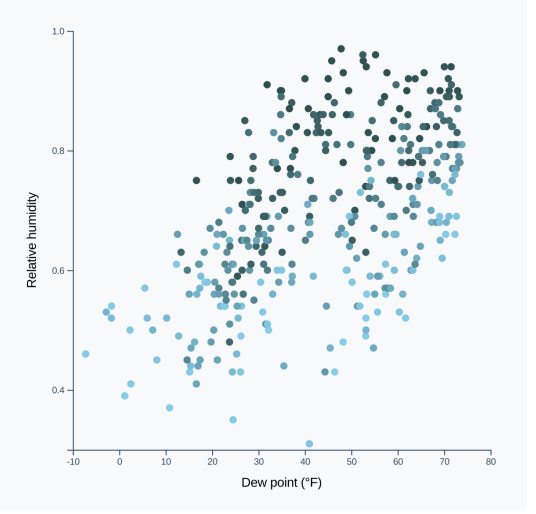
All that's left to do is to update how we set the **fill** of each dot. Let's find where we're doing that now.

```
.attr("fill", "cornflowerblue")
```

Instead of making every dot blue, let's use our `colorAccessor()` to grab the precipitation value, then pass that into our `colorScale()`.

```
.attr("fill", d => colorScale(colorAccessor(d)))
```

When we refresh our webpage, we should see our finished scatter plot with dots of various blues.



Scatterplot with a color scale

> For a complete, accessible chart, it would be a good idea to add a legend to explain what our colors mean. Stay tuned! We'll learn how to add a color scale legend in **Chapter 6**.

This chapter was jam-packed with new concepts — we learned about data joins, `<text>` SVG elements, color scales, and more. Give yourself a pat on the back for making it through! Next up, we'll create a bar chart and learn some new concepts.

Making a Bar Chart

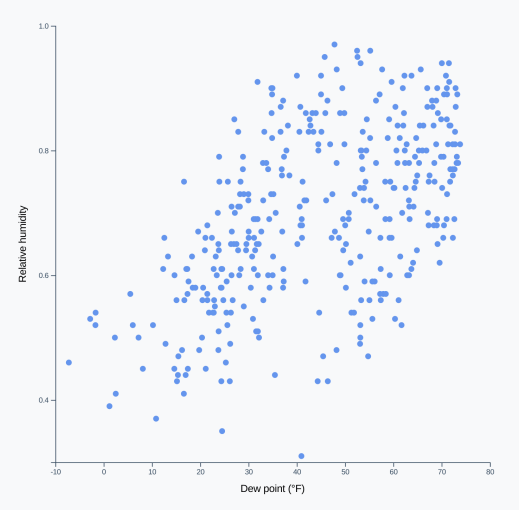
We'll walk through one last "basic" chart — once finished, you'll feel very comfortable with each step and we'll move on to even more exciting concepts like animations and interactions.

Deciding the chart type

Another type of question that we can ask our dataset is: what does the *distribution* of a metric look like? For example:

- What kinds of humidity values do we have?
- Does the humidity level generally stay around one value, with a few very humid days and a few very dry days?
- Or does it vary consistently, with no standard value?

Looking at the scatter plot we just made, we can see the daily humidity values from the dots' vertical placement.



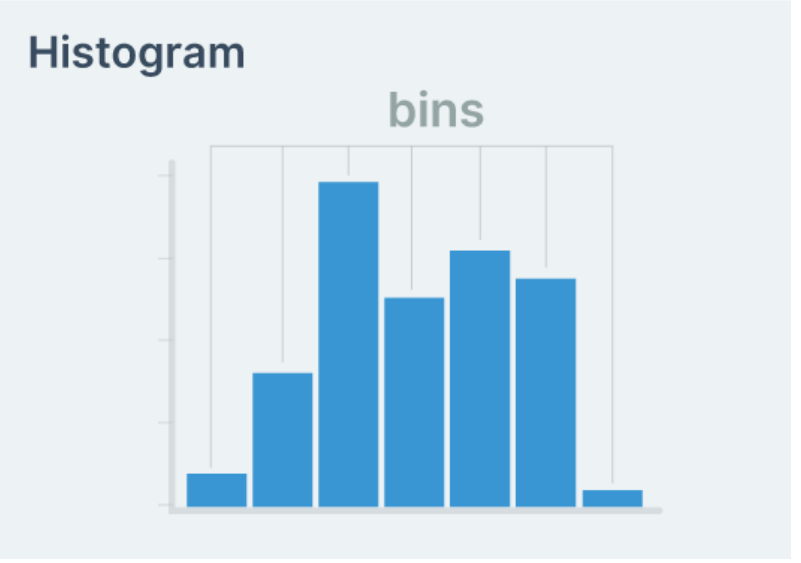
Finished scatter plot

But it's hard to answer our questions - do most of our dots fall close the middle of the chart? We're not entirely sure.

Instead, let's make a histogram.

Histogram

A *histogram* is a bar chart that shows the *distribution* of one metric, with the metric **values** on the x axis and the **frequency of values** on the y axis.



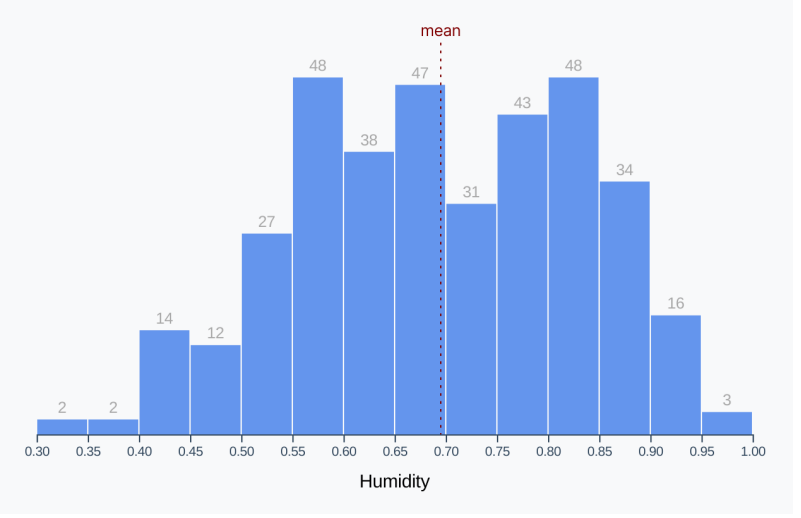
Histogram graphic

In order to show the frequency, **values are placed in equally-sized bins** (visualized as individual bars). For example, we could make bins for dew point temperatures that span 10 degrees - these would look something like `[0-10, 10-20, 20-30, ...]`. A dew point of 15 degrees would be counted in the second bin: `10-20`.

The number of and size of bins is up to the implementor - you could have a histogram with only 3 bins or one with 100 bins! There are standards that can be followed (feel free to check out d3's built-in formulas[22]), but we can generally decide the number based on what suits the data and what's easy to read.

Our goal is to make a histogram of humidity values. This will show us the distribution of humidity values and help answer our questions. Do most days stay around the same level of humidity? Or are there two types of days: humid and dry? Are there crazy humid days?

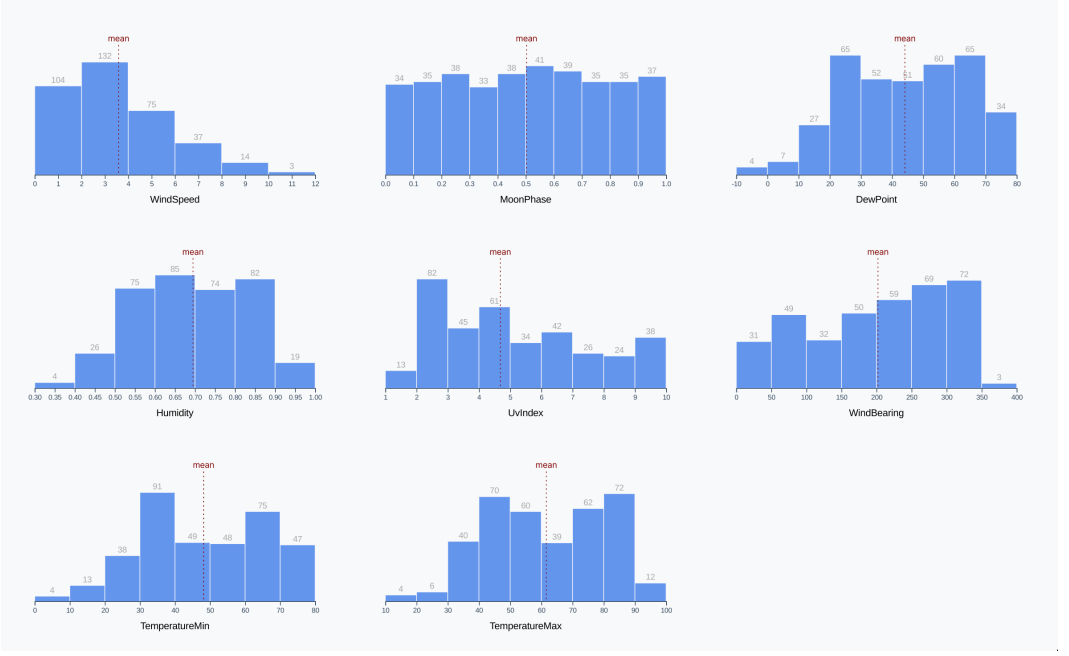
[22] https://github.com/d3/d3-array#bin-thresholds



Finished humidity histogram

To interpret the above histogram, it shows that we have **48** days in our dataset with a humidity value between `0.55` and `0.6`

For extra credit, we'll generalize our histogram function and loop through eight metrics in our dataset - creating many histograms to compare!



Many histograms

Let's dig in.

Chart checklist

To start, let's look over our chart-making checklist to remind ourselves of the necessary steps.

1. **Access data**
2. **Create dimensions**
3. **Draw canvas**
4. **Create scales**
5. **Draw data**
6. **Draw peripherals**
7. **Set up interactions**

We'll breeze through most of these steps, reinforcing what we've already learned.

Access data

As usual, make sure your node server is running (`live-server`) and point your browser at http://localhost:8080/03-making-a-bar-chart/draft/. Inside the

`/code/03-making-a-bar-chart/draft/draw-bars.js`

file, let's grab the data from our JSON file, waiting until it's loaded to continue.

code/03-making-a-bar-chart/completed/draw-bars.js

```
4   const dataset = await d3.json("./../../my_weather_data.json")
```

> As usual, the completed chart code is available if you need a hint, this time at `/code/03-making-a-bar-chart/completed/draw-bars.js`.

This time, **we're only interested in one metric for the whole chart**. Remember, the y axis is plotting the *frequency* (i.e. the number of occurrences) of the metric whose values are on the x axis. So instead of an `xAccessor()` and `yAccessor()`, we define a **single** `metricAccessor()`.

```
const metricAccessor = d => d.humidity
```

Create dimensions

Histograms are easiest to read when they are wider than they are tall. Let's set the width before defining the rest of our dimensions so we can use it to calculate the height. We'll also be able to quickly change the `width` later and keep the same aspect ratio for our chart.

Chart design tip: Histograms are easiest to read when they are wider than they are tall.

Instead of filling the whole window, let's prepare for multiple histograms and keep our chart small. That way, the charts can stack horizontally and vertically, depending on the screen size.

code/03-making-a-bar-chart/completed/draw-bars.js

```
11  const width = 600
```

Alright! Let's use the `width` to set the `width` and `height` of our chart. We'll leave a larger margin on the top to account for the bar labels, which we'll position above each bar.

code/03-making-a-bar-chart/completed/draw-bars.js

```
12  let dimensions = {
13    width: width,
14    height: width * 0.6,
15    margin: {
16      top: 30,
17      right: 10,
18      bottom: 50,
19      left: 50,
20    },
21  }
```

Remember, our **wrapper** encompasses the whole chart. If we subtract our margins, we'll get the size of our **bounds** which contain any data elements.

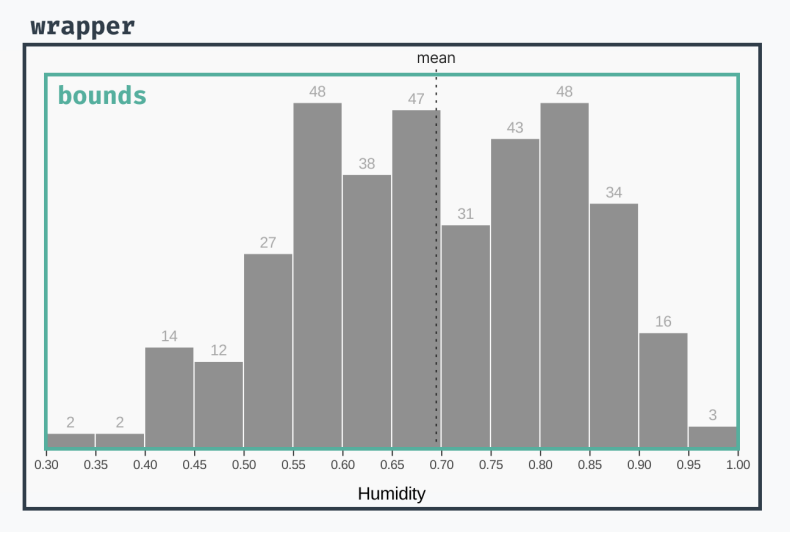


Chart terminology

Now that we know the size of our **wrapper** and **margins**, we can calculate the size of our **bounds**.

code/03-making-a-bar-chart/completed/draw-bars.js

```
  dimensions.boundedWidth = dimensions.width
    - dimensions.margin.left
    - dimensions.margin.right
  dimensions.boundedHeight = dimensions.height
    - dimensions.margin.top
    - dimensions.margin.bottom
```

Draw canvas

Let's create our **wrapper** element. Try to write this code without looking first. Like we've done before, we want to select the existing element, add a new `<svg>` element, and set its **width** and **height**.

code/03-making-a-bar-chart/completed/draw-bars.js

```
31  const wrapper = d3.select("#wrapper")
32    .append("svg")
33      .attr("width", dimensions.width)
34      .attr("height", dimensions.height)
```

How far did you get without looking? Let's try that again for this next part: creating our **bounds**. As a reminder, our **bounds** are a `<g>` element that will contain our main chart bits and be shifted to respect our **top** and **left** margins.

code/03-making-a-bar-chart/completed/draw-bars.js

```
36  const bounds = wrapper.append("g")
37      .style("transform", `translate(${
38        dimensions.margin.left
39      }px, ${
40        dimensions.margin.top
41      }px)`)
```

Perfect! Let's make our scales.

Create scales

Our x scale should look familiar to the ones we've made in the past. We need a scale that will convert humidity levels into pixels-to-the-right. Since both the **domain** and the **range** are continuous numbers, we'll use our friend `d3.scaleLinear()`.

Let's also use `.nice()`, which we learned in **Chapter 2**, to make sure our axis starts and ends on round numbers.

code/03-making-a-bar-chart/completed/draw-bars.js

```
45  const xScale = d3.scaleLinear()
46    .domain(d3.extent(dataset, metricAccessor))
47    .range([0, dimensions.boundedWidth])
48    .nice()
```

Rad. Now we need to create our `yScale`.

But wait a minute! We can't make a y scale without knowing the range of frequencies we need to cover. Let's create our data bins first.

Creating Bins

How can we split our data into bins, and what size should those bins be? We could do this manually by looking at the domain and organizing our days into groups, but that sounds tedious.

Thankfully, we can use **d3-array**'s `d3.histogram()` method to create a bin generator. This generator will convert our dataset into an array of bins - we can even choose how many bins we want!

Let's create a new generator:

code/03-making-a-bar-chart/completed/draw-bars.js

```
50  const binsGenerator = d3.histogram()
```

Similar to making a scale, we'll pass a **domain** to the generator to tell it the range of numbers we want to cover.

code/03-making-a-bar-chart/completed/draw-bars.js

```
50  const binsGenerator = d3.histogram()
51    .domain(xScale.domain())
```

Next, we'll need to tell our generator how to get the the **humidity** value, since our dataset contains objects instead of values. We can do this by passing our `metricAccessor()` function to the `.value()` method.

code/03-making-a-bar-chart/completed/draw-bars.js

```
50  const binsGenerator = d3.histogram()
51    .domain(xScale.domain())
52    .value(metricAccessor)
```

We can also tell our generator that we want it to aim for a specific number of bins. When we create our bins, we won't necessarily get this exact amount, but it should be close.

Let's aim for **13** bins — this should make sure we have enough granularity to see the shape of our distribution without too much noise. Keep in mind that the number of bins is the number of **thresholds + 1**.

code/03-making-a-bar-chart/completed/draw-bars.js

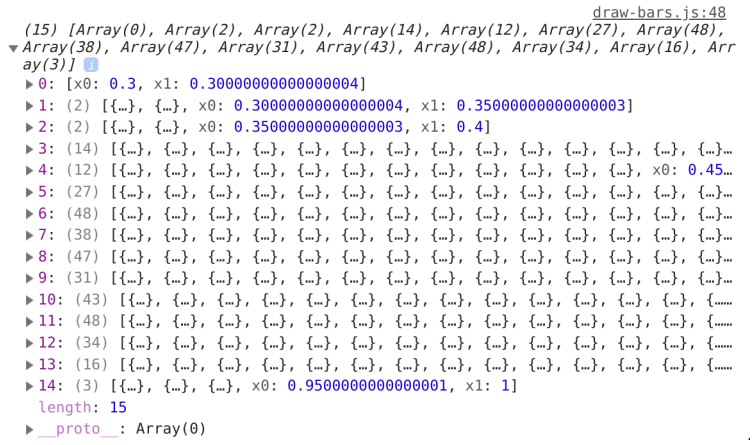
```
50  const binsGenerator = d3.histogram()
51    .domain(xScale.domain())
52    .value(metricAccessor)
53    .thresholds(12)
```

Great! Our bin generator is ready to go. Let's create our bins by feeding it our data.

code/03-making-a-bar-chart/completed/draw-bars.js

```
55  const bins = binsGenerator(dataset)
```

Let's take a look at these bins by logging them to the console: `console.log(bins)`.



logged bins

Each bin is an array with the following structure:

- each item is a matching data point. For example, the first bin has no matching days — this is likely because we used `.nice()` to round out our x scale.
- there is an **x0** key that shows the lower bound of included humidity values (inclusive)
- there is an **x1** key that shows the upper bound of included humidity values (exclusive). For example, a bin with a **x1** value of **1** will include values **up to 1**, but not **1** itself

> Note how there are 15 bins in my example — our bin generator was aiming for 13 bins but decided that 15 bins were more appropriate. This was a good decision, creating bins with a sensible size of `0.05`. If our bin generator had been more strict about the number of bins, our bins would have ended up with a size of `0.06666667`, which is harder to reason about. To extract insights from a chart, readers will mentally convert awkward numbers into rounder numbers to make sense of them. Let's do that work for them.

If we want, we can specify an exact number of bins by instead passing an array of **thresholds**. For example, we could specify 5 bins with `.thresholds([0, 0.2, 0.4, 0.6, 0.8, 1])`.

Creating the y scale

Okay great, now we can use these bins to create our y scale. First, let's create a y accessor function and throw it at the top of our file. Now that we know the shape of the data that we'll use to create our data elements, we can specify how to access the y value in one place.

code/03-making-a-bar-chart/completed/draw-bars.js

```
7   const yAccessor = d => d.length
```

Let's use our new accessor function and our bins to create that y scale. As usual, we'll want to make a linear scale. This time, however, **we'll want to start our y axis at zero**.

Previously, we wanted to represent the extent of our data since we were plotting metrics that had no logical bounds (temperature and humidity level). But the number of days that fall in a bin is bounded at `0` — you can't have negative days in a bin!

Instead of using `d3.extent()`, we can use another method from **d3-array**: `d3.max()`. This might sound familiar — we've used its counterpart, `d3.min()` in **Chapter 2**. `d3.max()` takes the same arguments: an array and an accessor function.

Note that we're passing `d3.max()` our `bins` instead of our original `dataset` — we want to find the maximum number of days in a bin, which is only available in our computed `bins` array.

code/03-making-a-bar-chart/completed/draw-bars.js

```
57  const yScale = d3.scaleLinear()
58    .domain([0, d3.max(bins, yAccessor)])
59    .range([dimensions.boundedHeight, 0])
```

Let's use `.nice()` here as well to give our bars a round top number.

code/03-making-a-bar-chart/completed/draw-bars.js

```
57  const yScale = d3.scaleLinear()
58    .domain([0, d3.max(bins, yAccessor)])
59    .range([dimensions.boundedHeight, 0])
60    .nice()
```

Draw data

Here comes the fun part! Our plan is to create one bar for each bin, with a label on top of each bar.

We'll need one bar for each item in our `bins` array — this is a sign that we'll want to use the **data bind** concept we learned in **Chapter 2**.

Let's first create a `<g>` element to contain our bins. This will help keep our code organized and isolate our bars in the DOM.

code/03-making-a-bar-chart/completed/draw-bars.js

```
64  const binsGroup = bounds.append("g")
```

Because we have more than one element, we'll bind each data point to a `<g>` SVG element. This will let us group each bin's **bar** and **label**.

To start, we'll select all existing `<g>` elements within our `binsGroup` (*there aren't any yet, but we're creating a selection object that points to the right place*). Then we'll use `.data()` to bind our `bins` to the selection.

code/03-making-a-bar-chart/completed/draw-bars.js

```
66  const binGroups = binsGroup.selectAll("g")
67    .data(bins)
```

Next, we'll create our `<g>` elements, using `.enter()` to target the new bins (*hint: all of them*).

code/03-making-a-bar-chart/completed/draw-bars.js

```
66  const binGroups = binsGroup.selectAll("g")
67    .data(bins)
68    .enter().append("g")
```

The above code will create one new `<g>` element for each bin. We're going to place our bars within this group.

Next up we'll draw our bars, but first we should calculate any constants that we'll need. Like a warrior going into battle, we want to prepare our weapons before things heat up.

In this case, the only constant that we can set ahead of time is **the padding between bars**. Giving them some space helps distinguish individual bars, but we don't want them too far apart - that will make them hard to compare and take away from the overall shape of the distribution.

Chart design tip: putting a space between bars helps distinguish individual bars

code/03-making-a-bar-chart/completed/draw-bars.js

```
70  const barPadding = 1
```

Now we are armed warriors and are ready to charge into battle! Each bar is a rectangle, so we'll append a `<rect>` to each of our `<g>` elements.

code/03-making-a-bar-chart/completed/draw-bars.js

```
71  const barRects = binGroups.append("rect")
72      .attr("x", d => xScale(d.x0) + barPadding / 2)
73      .attr("y", d => yScale(yAccessor(d)))
```

Remember, `<rect>`s need four attributes: **x**, **y**, **width**, and **height**.

Let's start with the **x** value, which will corresponds to the *left* side of the bar. The bar will start at the lower bound of the bin, which we can find at the **x0** key.

But **x0** is a humidity level, not a pixel. So let's use `xScale()` to convert it to pixel space.

Lastly, we need to offset it by the `barPadding` we set earlier.

code/03-making-a-bar-chart/completed/draw-bars.js

```
72        .attr("x", d => xScale(d.x0) + barPadding / 2)
73        .attr("y", d => yScale(yAccessor(d)))
74        .attr("width", d => d3.max([
```

> We could create accessor functions for the **x0** and **x1** properties of each bin if we were concerned about the structure of our bins changing. In this case, it would be overkill since:
>
> 1. we didn't specify the structure of each bin, `d3.histogram()` did
> 2. we're not going to change the way we access either of these values since they're built in to `d3.histogram()`
> 3. the way we access these properties is very straightforward. If the values were more nested or required computation, we could definitely benefit from accessor functions.

Next, we'll specify the `<rect>`'s **y** attribute which corresponds to the top of the bar. We'll use our `yAccessor()` to grab the frequency and use our scale to convert it into pixel space.

code/03-making-a-bar-chart/completed/draw-bars.js

```
73        .attr("y", d => yScale(yAccessor(d)))
74        .attr("width", d => d3.max([
75          0,
```

To find the width of a bar, we need to **subtract the x0 position of the left side of the bar from the x1 position of the right side of the bar**.

We'll need to subtract the bar padding from the total width to account for spaces between bars. Sometimes we'll get a bar with a `width` of `0`, and subtracting the

`barPadding` will bring us to `-1`. To prevent passing our `<rect>`s a negative `width`, we'll wrap our value with `d3.max([0, width])`.

code/03-making-a-bar-chart/completed/draw-bars.js

```
74        .attr("width", d => d3.max([
75          0,
76          xScale(d.x1) - xScale(d.x0) - barPadding
77        ]))
78        .attr("height", d => dimensions.boundedHeight
79          - yScale(yAccessor(d))
```

Lastly, we'll calculate the bar's height by subtracting the **y** value from the bottom of the y axis. Since our y axis starts from 0, we can use our `boundedHeight`.

code/03-making-a-bar-chart/completed/draw-bars.js

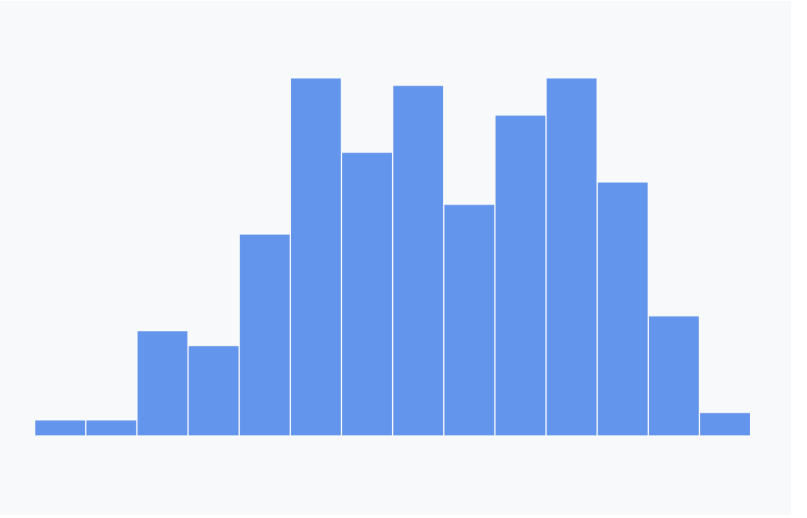
```
78        .attr("height", d => dimensions.boundedHeight
79          - yScale(yAccessor(d))
80        )
```

Let's put that all together and change the bar **fill** to blue.

code/03-making-a-bar-chart/completed/draw-bars.js

```
71    const barRects = binGroups.append("rect")
72        .attr("x", d => xScale(d.x0) + barPadding / 2)
73        .attr("y", d => yScale(yAccessor(d)))
74        .attr("width", d => d3.max([
75          0,
76          xScale(d.x1) - xScale(d.x0) - barPadding
77        ]))
78        .attr("height", d => dimensions.boundedHeight
79          - yScale(yAccessor(d))
80        )
81        .attr("fill", "cornflowerblue")
```

Alright! When we refresh our webpage, we'll see the beginnings of our histogram!



Our bars

Adding Labels

Let's add labels to show the count for each of these bars.

We can keep our chart clean by only adding labels to bins with any relevant days — having `0`s in empty spaces is unhelpful visual clutter. We can identify which bins have no data by their lack of a bar, no need to call it out with a label.

d3 selections have a `.filter()` method that acts the same way the native Array method does. `.filter()` accepts one parameter: a function that accepts one data point and returns a value. Any items in our dataset who return a **falsy** value will be removed.

> By "**falsy**", we're referring to any value that evaluates to `false`. Maybe surprisingly, this includes values other than `false`, such as `0`, `null`, `undefined`, `""`, and `NaN`. Keep in mind that empty arrays `[]` and object `{}` evaluate to **truthy**. If you're

curious, read more here[a].

[a] https://developer.mozilla.org/en-US/docs/Glossary/Falsy

We can use `yAccessor()` as shorthand for `d => yAccessor(d) != 0` because `0` is **falsy**.

code/03-making-a-bar-chart/completed/draw-bars.js

```
83  const barText = binGroups.filter(yAccessor)
```

Since these labels are just text, we'll want to use the SVG `<text>` element we've been using for our axis labels.

code/03-making-a-bar-chart/completed/draw-bars.js

```
83  const barText = binGroups.filter(yAccessor)
84    .append("text")
```

Remember, `<text>` elements are positioned with **x** and **y** attributes. The label will be centered horizontally above the bar — we can find the center of the bar by adding half of the bar's width (*the right side minus the left side*) to the left side of the bar.

code/03-making-a-bar-chart/completed/draw-bars.js

```
83  const barText = binGroups.filter(yAccessor)
84    .append("text")
85      .attr("x", d => xScale(d.x0) + (xScale(d.x1) - xScale(d.x0)) / 2)
```

Our `<text>`'s y position will be similar to the `<rect>`'s y position, but let's shift it up by 5 pixels to add a little gap.

code/03-making-a-bar-chart/completed/draw-bars.js

```
83    const barText = binGroups.filter(yAccessor)
84      .append("text")
85        .attr("x", d => xScale(d.x0) + (xScale(d.x1) - xScale(d.x0)) / 2)
86        .attr("y", d => yScale(yAccessor(d)) - 5)
```

Next, we'll display the count of days in the bin using our `yAccessor()` function. *Note: again, we can use* `yAccessor()` *as shorthand for* `d => yAccessor(d)`.

code/03-making-a-bar-chart/completed/draw-bars.js

```
83    const barText = binGroups.filter(yAccessor)
84      .append("text")
85        .attr("x", d => xScale(d.x0) + (xScale(d.x1) - xScale(d.x0)) / 2)
86        .attr("y", d => yScale(yAccessor(d)) - 5)
87        .text(yAccessor)
```

We can use the CSS `text-anchor` property to horizontally align our text — this is a much simpler solution than compensating for text width when we set the **x** attribute.

code/03-making-a-bar-chart/completed/draw-bars.js

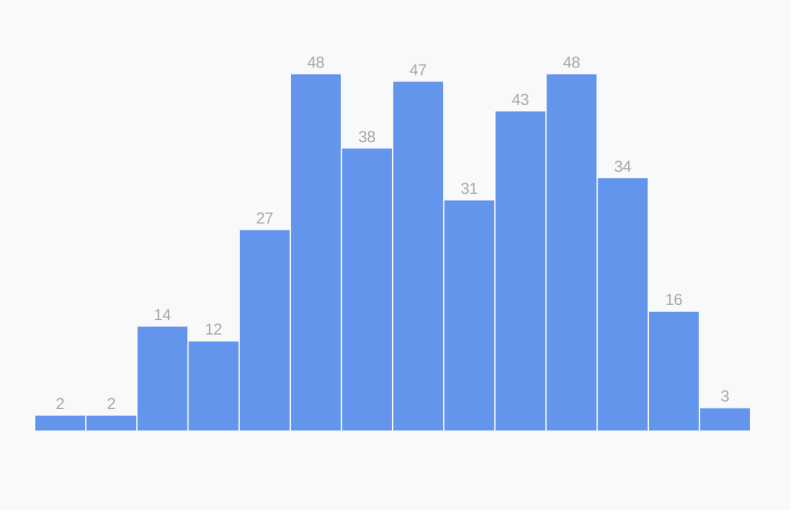
```
83    const barText = binGroups.filter(yAccessor)
84      .append("text")
85        .attr("x", d => xScale(d.x0) + (xScale(d.x1) - xScale(d.x0)) / 2)
86        .attr("y", d => yScale(yAccessor(d)) - 5)
87        .text(yAccessor)
88        .style("text-anchor", "middle")
```

After adding a few styles to decrease the visual importance of our labels…

code/03-making-a-bar-chart/completed/draw-bars.js

```
83    const barText = binGroups.filter(yAccessor)
84      .append("text")
85        .attr("x", d => xScale(d.x0) + (xScale(d.x1) - xScale(d.x0)) / 2)
86        .attr("y", d => yScale(yAccessor(d)) - 5)
87        .text(yAccessor)
88        .style("text-anchor", "middle")
89        .attr("fill", "darkgrey")
90        .style("font-size", "12px")
91        .style("font-family", "sans-serif")
```

...we should see the count of days for each of our bars!



Our bars with labels

Extra credit

When looking at the shape of a distribution, it can be helpful to know where the mean is.

The mean is just the average — the center of a set of numbers. To calculate the mean, you would divide the sum by the number of values. For example, the mean of `[1, 2, 3, 4, 5]` would be `(1 + 2 + 3 + 4 + 5) / 5 = 3`.

Instead of calculating the mean by hand, we can use `d3.mean()` to grab that value. Like many d3 methods we've used, we pass the dataset as the first parameter and an optional accessor function as the second.

code/03-making-a-bar-chart/completed/draw-bars.js

```
93  const mean = d3.mean(dataset, metricAccessor)
```

Great! Let's see how comfortable we are with drawing an unfamiliar SVG element: `<line>`. A `<line>` element will draw a line between two points: `[x1, y1]` and `[x2, y2]`. Using this knowledge, let's add a line to our bounds that is:

- at the mean humidity level,
- starting 15px above our chart, and
- ending at our x axis.

How close can you get before looking at the following code?

code/03-making-a-bar-chart/completed/draw-bars.js

```
94  const meanLine = bounds.append("line")
95      .attr("x1", xScale(mean))
96      .attr("x2", xScale(mean))
97      .attr("y1", -15)
98      .attr("y2", dimensions.boundedHeight)
```

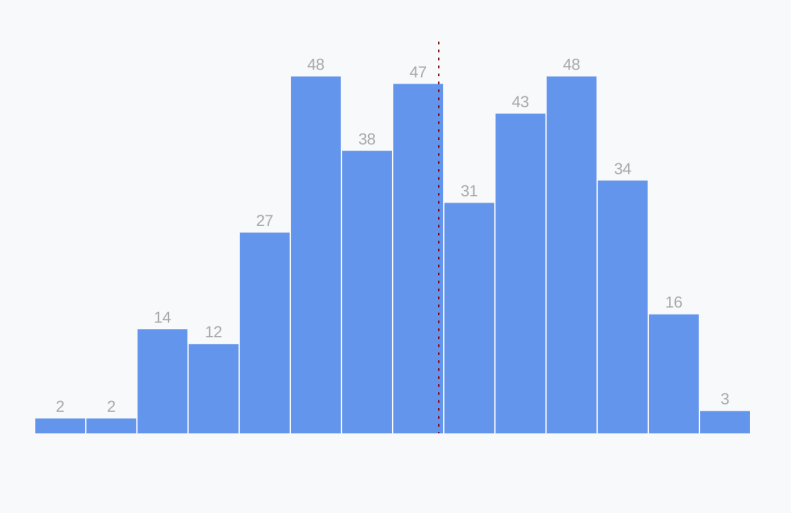
Let's add some styles to the line so we can see it (by default, `<line>`s have no stroke color) and to distinguish it from an axis. SVG element strokes can be split into dashes with the `stroke-dasharray` attribute. The lines alternate between the stroke color and transparent, starting with transparent. We define the line lengths with a space-separated list of values (which will be repeated until the line is drawn).

Let's make our lines dashed with a 2px long maroon dash and a 4px long gap.

code/03-making-a-bar-chart/completed/draw-bars.js

```
94    const meanLine = bounds.append("line")
95        .attr("x1", xScale(mean))
96        .attr("x2", xScale(mean))
97        .attr("y1", -15)
98        .attr("y2", dimensions.boundedHeight)
99        .attr("stroke", "maroon")
100       .attr("stroke-dasharray", "2px 4px")
```

Give yourself a pat on the back for drawing your first `<line>` element!



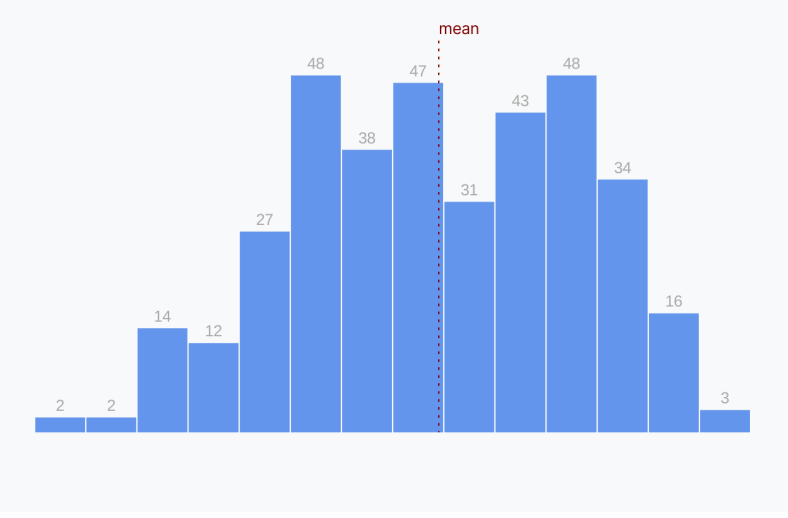
Our bars with labels and the mean

Let's label our line to clarify to readers what it represents. We'll want to add a `<text>` element in the same position as our line, but 5 pixels higher to give a little gap.

code/03-making-a-bar-chart/completed/draw-bars.js

```
102   const meanLabel = bounds.append("text")
103       .attr("x", xScale(mean))
104       .attr("y", -20)
105       .text("mean")
106       .attr("fill", "maroon")
107       .style("font-size", "12px")
```

Hmm, we can see the text but it isn't horizontally centered with our line.



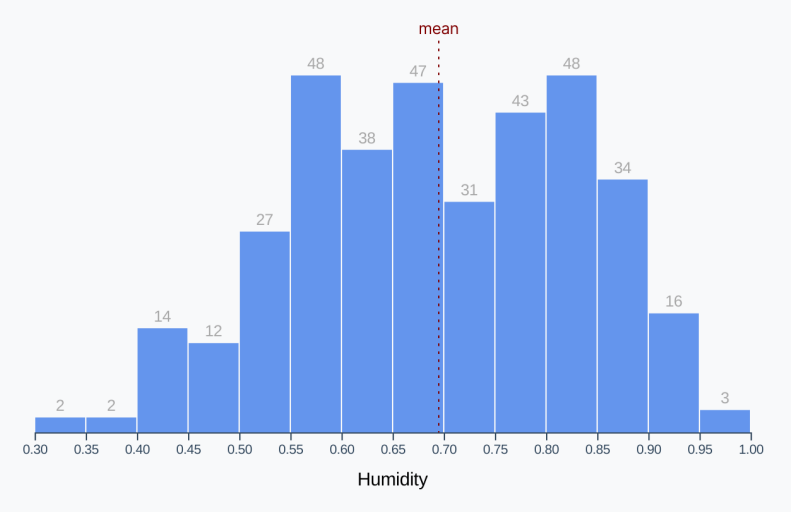
Our bars with a mean label

Let's center our text by adding the CSS property `text-anchor: middle`. This is a property specifically for setting the horizontal alignment of text in SVG.

code/03-making-a-bar-chart/completed/draw-bars.js

```
102   const meanLabel = bounds.append("text")
103       .attr("x", xScale(mean))
104       .attr("y", -20)
105       .text("mean")
106       .attr("fill", "maroon")
107       .style("font-size", "12px")
108       .style("text-anchor", "middle")
```

Perfect! Now our mean line is clear to our readers.



Our bars with a mean label, centered horizontally

Draw peripherals

As usual, our last task here is to draw our axes. But we're in for a treat! Since we're labeling the y value of each of our bars, we won't need a y axis. We just need an x axis and we're set!

We'll start by making our axis generator — our axis will be along the bottom of the chart so we'll be using `d3.axisBottom()`.

code/03-making-a-bar-chart/completed/draw-bars.js

```
112   const xAxisGenerator = d3.axisBottom()
113     .scale(xScale)
```

Then we'll use our new axis generator to create an axis, then shift it below our bounds.

code/03-making-a-bar-chart/completed/draw-bars.js

```
115   const xAxis = bounds.append("g")
116     .call(xAxisGenerator)
117       .style("transform", `translateY(${dimensions.boundedHeight}px)`)
```

And lastly, let's throw a label on there to make it clear what the tick labels represent.

```
const xAxisLabel = xAxis.append("text")
    .attr("x", dimensions.boundedWidth / 2)
    .attr("y", dimensions.margin.bottom - 10)
    .attr("fill", "black")
    .style("font-size", "1.4em")
    .text("Humidity")
```

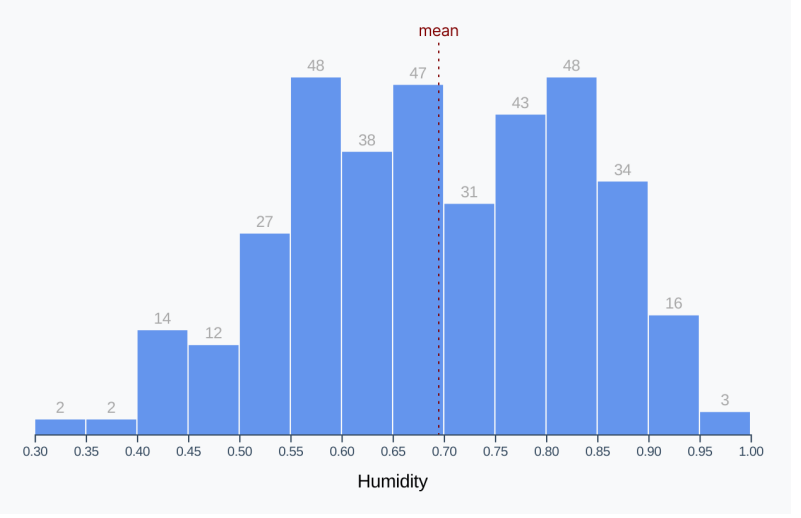
And voila, we're done drawing our peripherals!

Set up interactions

Next, we would set up any chart interactions. We don't have any interactions for this chart, but stay tuned — we'll cover this in the next chapter.

Looking at our chart

Chart finished! Let's take a look at our distribution.



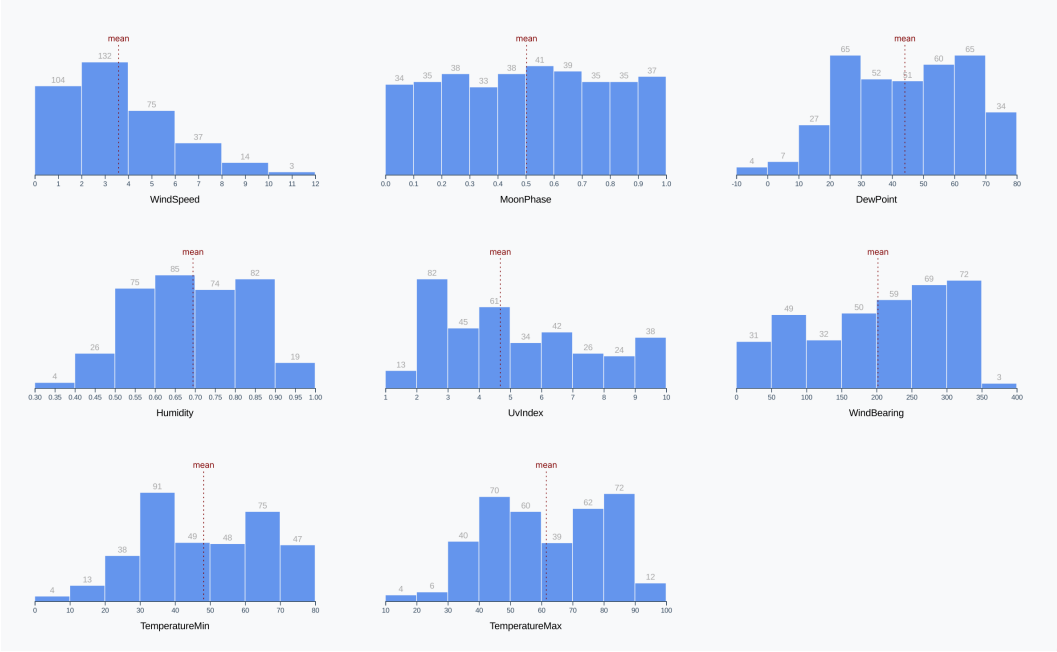
Finished humidity histogram

Our histogram looks somewhere in-between a normal and bimodal distribution. Don't worry if those terms make no sense right now — we'll cover distribution shapes in detail in **Chapter 8**. If you used your own weather data, your histogram could have a very different shape!

Extra credit

Let's generalize our histogram drawing function and create a chart for each weather metric we have access to! This will make sure that we understand what every line of code is doing.

Generalizing our code will also help us to start thinking about handling dynamic data — a core concept when building a dashboard. Drawing a graph with a specific dataset can be difficult, but you get to rely on values being the same every time your code runs. When handling data from an API, your charting functions need to be more robust and able to handle very different datasets.



Finished histogram

Here's the good news: we won't need to rewrite the majority of our code! The main difference is that we'll wrap most of the chart drawing into a new function called `drawHistogram()`.

Which steps do we need to repeat for every chart? Let's look at our checklist again.

1. **Access data**
2. **Create dimensions**
3. **Draw canvas**
4. **Create scales**
5. **Draw data**
6. **Draw peripherals**
7. **Set up interactions**

All of the histograms will use the same dataset, so we can skip step **1**. And every chart will be the same size, so we don't need to repeat step **2** either. However, we want each chart to have its own **svg** element, so we'll need to wrap everything after step **2** .

> In the next section, we'll cover ways to make our chart more accessible. We'll be working on the current version of our histogram - make a copy of your current finished histogram in order to come back to it later.

Let's do that — we'll create a new function called `drawHistogram()` that contains all of our code, starting at the point we create our svg. Note that the finished code for this step is in the `/code/03-making-a-bar-chart/completed-multiple/draw-bars.js` file if you're unsure about any of these steps.

```
const drawHistogram = () => {
    const wrapper = d3.select("#wrapper")
    // ...  the rest of our chart code
```

What parameters does our function need? The only difference between these charts is the metric we're plotting, so let's add that as an argument.

```
const drawHistogram = metric => {
    // ...
```

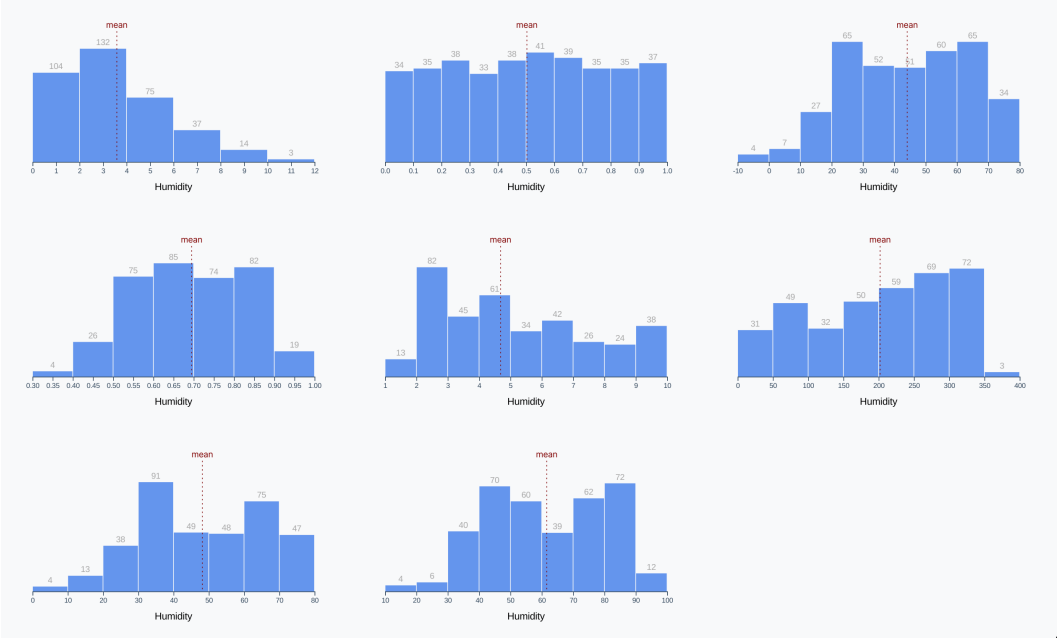
But wait, we need to use the metric to update our `metricAccessor()`. Let's grab our accessor functions from our **Access data** step and throw them at the top of our new function. We'll also need our `metricAccessor()` to return the provided metric, instead of hard-coding `d.humidity`.

```
const drawHistogram = metric => {
    const metricAccessor = d => d[metric]
    const yAccessor = d => d.length

    const wrapper = d3.select("#wrapper")
    // ...
```

Great, let's give it a go! At the bottom of our `drawBars()` function, let's run through some of the available metrics (see code example for a list) and pass each of them to our new generalized function.

```
const metrics = [
    "windSpeed",
    "moonPhase",
    "dewPoint",
    "humidity",
    "uvIndex",
    "windBearing",
    "temperatureMin",
    "temperatureMax",
]

metrics.forEach(drawHistogram)
```

Alright! Let's see what happens when we refresh our webpage.



Finished histograms, wrong labels

We see multiple histograms, but something is off. Not all of these charts are showing **Humidity**! Let's find the line where we set our x axis label and update that to show our metric instead. Here it is:

```
const xAxisLabel = xAxis.append("text")
    //  ...
     .text("Humidity")
```

We'll set the text to our metric instead, and we can also add a CSS `text-transform` value to help format our metric names. For a production dashboard, we might want to look up a proper label in a metric-to-label map, but this will work in a pinch.

```
const xAxisLabel = xAxis.append("text")
     // ...
     .text(metric)
     .style("text-transform", "capitalize")
```

When we refresh our webpage, we should see our finished histograms.

Finished histogram

Wonderful!

Take a second and observe the variety of shapes of these histograms. What are some insights we can discover when looking at our data in this format?

- the **moon phase** distribution is flat - this makes sense because it's cyclical, consistently going through the same steps all year.
- our **wind speed** is usually around 3 mph, with a long tail to the right that represents a few very windy days. Some days have no wind at all, with an average wind speed of 0.
- our **max temperatures** seem almost bimodal, with the mean falling in between two humps. Looks like New York City spends more days with relatively extreme temperatures (30°F - 50°F or 70°F - 90°F) than with more temperate weather (60°F).

Accessibility

The main goal of any data visualization is for it to be readable. This generally means that we want our elements to be easy to see, text is large enough to read, colors have enough contrast, etc. But what about users who access web pages through screen readers?

We can actually make our charts accessibile at a basic level, without putting a lot of effort in. Let's update our histogram so that it's accessible with a screen reader.

If you want to test this out, download the ChromeVox[23] extension for chrome (or use any other screen reader application). If we test it out on our histogram, you'll notice that it doesn't give much information, other than reading all of the text in our chart. That's not an ideal experience.

The main standard for making websites accessible is from **WAI-ARIA**, the Accessible Rich Internet Applications Suite. WAI-ARIA roles, set using a `role` attribute, tell the screen reader what *type of content* an element is.

> We'll be updating our completed *single* histogram in this section. If you completed the previous **Extra credit** section, either find your backup of your code or use the completed code in the `/03-making-a-bar-chart/completed/` folder.

[23] https://chrome.google.com/webstore/detail/chromevox/kgejglhpjiefppelpmljglcjbhoiplfn?hl=en

The first thing we can do is to give our `<svg>` element a `role` of `figure`[24], to alert it that this element is a chart. (This code can go at the bottom of the **Draw canvas** step).

```
wrapper.attr("role", "figure")
```

Next, we can make our chart *tabbable*, by adding a `tabindex` of `0`. This will make it so that a user can hit `tab` to highlight our chart.

> There are only two `tabindex` values that you should use:
>
> 1. `0`, which puts an element in the **tab** flow, in DOM order
> 2. `-1`, which takes an element out of the **tab** flow.

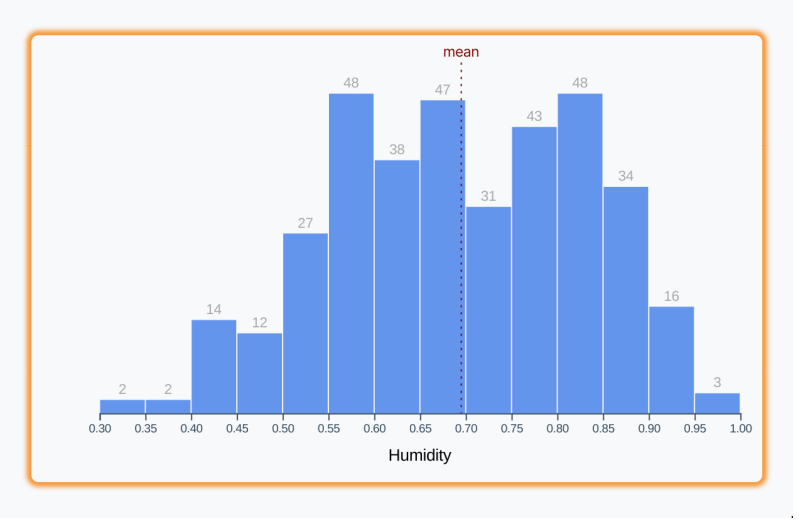
```
wrapper.attr("role", "figure")
    .attr("tabindex", "0")
```

When a user *tabs* to our chart, we want the screen reader to announce the basic layout so the user knows what they're "looking" at. To do this, we can add a `<title>` SVG component with a short description.

```
wrapper.append("title")
    .text("Histogram looking at the distribution of humidity in 2016")
```

If you have a screen reader set up, you'll notice that it will read our `<title>` when we tab to our chart. The "highlighted" state will look something like this:

[24] https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Roles/Figure_Role



Accessibility highlight

Next, we'll want to make our `binsGroup` selectable by also giving it a `tabindex` of `0`. If the user presses `tab` after the `wrapper` is focused, the browser will focus on the `binsGroup` because it's the next element (in DOM order) that is focusable.

```
const binsGroup = bounds.append("g")
    .attr("tabindex", "0")
```

We can also give our `binsGroup` a `role` of `"list"`, which will make the screen reader announce the number of items within the list. And we'll let the user know *what* the list contains by adding an `aria-label`.

```
const binsGroup = bounds.append("g")
    .attr("tabindex", "0")
    .attr("role", "list")
    .attr("aria-label", "histogram bars")
```

Now when our `binsGroup` is highlighted, the screen reader will announce: "histogram bars. List with 15 items". Perfect!

Let's annotate each of our "list items". After we create our `binGroups`, we'll add a few attributes to each group:

1. make it focusable with a `tabindex` of `0`
2. give it a `role` of `"listitem"`
3. give it an `aria-label` that the screen reader will announce when the item is *focused.*

```
const binGroups = binsGroup.selectAll("g")
  .data(bins)
  .enter().append("g")
    .attr("tabindex", "0")
    .attr("role", "listitem")
    .attr("aria-label", d => `There were ${
      yAccessor(d)
    } days between ${
      d.x0.toString().slice(0, 4)
    } and ${
      d.x1.toString().slice(0, 4)
    } humidity levels.`)
```

Now when we *tab* out of our `binsGroup`, it will focus the first bar group (and subsequent ones when we **tab**) and announce our aria label.

We'll tackle one last issue — you might have noticed that the screen reader reads each of our x-axis tick labels once it's done reading our `<title>`. This is pretty annoying, and not giving the user much information. Let's prevent that.

At the bottom of our `drawBars()` function, let's select all of the text within our chart and give it an `aria-hidden` attribute of `"true"`.

```
wrapper.selectAll("text")
   .attr("role", "presentation")
   .attr("aria-hidden", "true")
```

Great! Now our screen reader will read only our labels and ignore any `<text>` elements within our chart.

With just a little effort, we've made our chart accessible to any users who access the web through a screen reader. That's wonderful, and more than most online charts can say!

Next up, we'll get fancy with animations and transitions.

Animations and Transitions

When we update our charts, we can animate elements from their old to their new positions. These animations can be visually exciting, but more importantly, they have functional benefits. When a bar animates from one height to another, the viewer has a better idea of whether it's getting larger or smaller. If we're animating several bars at once, we're drawing the viewer's eye to the bar making the biggest change because of its fast speed.

By analogy, imagine if track runners teleported from the start to the finish line. For one, it would be terribly boring to watch, but it would also be hard to tell who was fastest.

There are multiple ways we can animate changes in our graphs:

- using SVG **`<animate>`**
- using CSS **`transition`**
- using **`d3.transition()`**

Let's introduce each of these options.

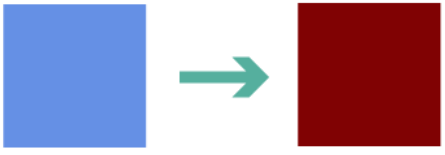
SVG `<animate>`

`<animate>` is a native SVG element that can be defined inside of the animated element.

```
<svg width="120" height="120">
  <rect x="10" y="10" width="100" height="100" fill="cornflowerblue">
    <animate
      attributeName="x"
      values="0;20;0"
      dur="2s"
      repeatCount="indefinite"
    />
    <animate
      attributeName="fill"
      values="cornflowerBlue;maroon;cornflowerBlue"
      dur="6s"
      repeatCount="indefinite"
    />
  </rect>
</svg>
```

With your server running (`live-server`), navigate to the

`4-animations-and-transitions/1-svg-animate/` folder in your browser to see this animation in action.



SVG animate

Unfortunately this won't work for our charts. For one, `<animate>` is **unsupported in Internet Explorer**. But the bigger issue is that we would have to set a static start and end value. We don't want to define static animations, instead we want our elements to animate changes between two dynamic values. Luckily, we have other options.

CSS transitions

Many of our chart changes can be transitioned with the CSS `transition` property. When we update a `<rect>` to have a fill of **red** instead of **blue**, we can specify that the color change take 10 seconds instead of being instantaneous. During those 10 seconds, the `<rect>` will continuously re-draw with intermediate colors on the way to **red**.

> Not all properties can be animated. For example, how would you animate changing a label from **Humidity** to **Dew point**? However most properties can be animated, so feel free to operate under the assumption that a property can be animated until proven otherwise.

Let's try out an example! Navigate to the

`4-animations-and-transitions/2-css-transition-playground/` folder in the browser. You'll see a blue box that moves and turns green on hover.

box transition

Let's open up the

`4-animations-and-transitions/2-css-transition-playground/styles.css` file to take a look at what's going on. We can see our basic styles for the box.

```
.box {
    background: cornflowerblue;
    height: 100px;
    width: 100px;
}
```

And our styles that apply to our box when it is hovered (change the background color and move it 30 pixels to the right).

```
.box:hover {
    background: yellowgreen;
    transform: translateX(30px);
}
```

To create CSS a transition, we need to specify how long we want the animation to take with the `transition-duration` property. The property value accepts **time CSS data types** — a number followed by either `s` (seconds) or `ms` (milliseconds).

Let's make our box changes animate over one second.

```
.box {
    background: cornflowerblue;
    height: 100px;
    width: 100px;
    transition-duration: 1s;
}
```

When we refresh our webpage and hover over the box, we can see it slowly move to the right and turn green. Smooth!

box transition all

Now let's say that we only want to animate our box's movement, but we want the color change to happen instantaneously. This is possible by specifying the `transition-property` CSS property. By default, `transition-property` is set to `all`, which animates all transitions. Instead, let's override the default and specify a specific CSS property name (`transform`).

```
.box {
    background: cornflowerblue;
    height: 100px;
    width: 100px;
    transition-duration: 1s;
    transition-property: transform;
}
```

Now our box instantly turns green, but still animates to the right.

box transition transform

Instead of setting `transition-duration` and `transition-property` separately, we can use the shorthand property: `transition`. Shorthand CSS properties let you set multiple properties in one line. When we give `transition` a CSS property name and duration (separated by a space), we are setting both `transition-duration` and `transition-property`. Let's try it out.

```
.box {
    background: cornflowerblue;
    height: 100px;
    width: 100px;
    transition: transform 1s;
}
```

`transition` will accept a third property (`transition-timing-function`) that sets the acceleration curve for the animation. The animation could be linear (the default), slow then fast (`ease-in`), in steps (`steps(6)`), or even a custom function (`cubic-bezier(0.1, 0.7, 1.0, 0.1)`), among other options. Let's see what `steps(6)` looks like — it should break the animation into 6 discrete steps.

```
.box {
    background: cornflowerblue;
    height: 100px;
    width: 100px;
    transition: transform 1s steps(6);
}
```

What if we wanted to animate the color change, but finish turning green *while* our box is shifting to the right? `transition` will accept multiple transition statements, we just need to separate them by a comma. Let's add a transition for the `background` color.

```
.box {
    background: cornflowerblue;
    height: 100px;
    width: 100px;
    transition: transform 1s steps(6),
                background 2s ease-out;
}
```

Nice! Now our box transitions by stepping to the right, while turning green over two seconds. Chrome's dev tools have a great way to visualize this transition. Press `esc` when looking at the **Elements** panel to bring up the bottom panel. In the bottom panel, we can open up the **Animations** tab.